

Chapter 4: The Network Layer: Data Plane

Questions and Solutions

1. 4.1 Overview of Network Layer

Question 1: Network-Layer Terminology

Recall that the name of a transport-layer packet is **segment** and that the name of a link-layer packet is **frame**.

1. What is the name of a network-layer packet?
2. What is the fundamental difference between a **router** and a **link-layer switch**?

Solution

- **Network-layer packet:** A network-layer packet is called a **datagram**.
- **Difference between Router and Switch:**
 - A **router** forwards a packet based on the packet's **IP (layer 3)** address.
 - A **link-layer switch** forwards a packet based on the packet's **MAC (layer 2)** address.

Question 2: Data Plane vs. Control Plane Functions

We noted that network layer functionality can be broadly divided into **data plane** functionality and **control plane** functionality. What are the main functions of the data plane? What are the main functions of the control plane?

Solution

1. Data Plane Functionality: The main function of the **data plane** is **packet forwarding**, which is to forward datagrams from their input links to their output links. Specific tasks include:

- Terminating incoming physical links (Physical layer function).
- Interoperating with the link layer on the other side of an incoming link.
- Performing the critical **lookup function** at the input ports to determine the output interface.

2. Control Plane Functionality: The main function of the **control plane** is **routing**, which is to determine the end-to-end paths a packet takes from source to destination. Responsibilities include:

- Executing **routing protocols** (e.g., OSPF, BGP).
- Responding to link-state changes (links going up or down).
- Communicating with remote controllers (in SDN environments).
- Performing various network management functions.

Question 3: Routing vs. Forwarding

We made a distinction between the **forwarding function** and the **routing function** performed in the network layer. What are the key differences between routing and forwarding?

Solution

The key differences between routing and forwarding can be categorized by their scope, timescale, and implementation:

- **Forwarding (Data Plane):**
 - **Scope:** A router's **local action** of transferring packets from its input interfaces to its output interfaces.
 - **Timescale:** Takes place at very **short timescales** (typically nanoseconds).
 - **Implementation:** Almost always implemented in **hardware**.
- **Routing (Control Plane):**
 - **Scope:** The **network-wide process** that determines the end-to-end paths that packets take from sources to destinations.
 - **Timescale:** Takes place on much **longer timescales** (typically seconds).
 - **Implementation:** Often implemented in **software**.

Question 4: The Role of the Forwarding Table

What is the role of the **forwarding table** within a router?

Solution

The role of the forwarding table within a router is to hold entries that determine the **outgoing link interface** to which an arriving packet will be forwarded via the **switching fabric**.

Question 5: Internet Service Model

We said that a network layer's service model "defines the characteristics of end-to-end transport of packets between sending and receiving hosts."

1. What is the service model of the Internet's network layer?
2. What guarantees are made by the Internet's service model regarding the host-to-host delivery of datagrams?

Solution

- **Service Model:** The service model of the Internet's network layer is **best-effort service**.
- **Guarantees:** With this service model, there are **no guarantees** regarding several key performance metrics:
 - No guarantee that packets will be received in the **order** in which they were sent.
 - No guarantee of their **eventual delivery**.
 - No guarantee on the **end-to-end delay**.
 - No **minimal bandwidth** guarantee.

2. 4.2 What's Inside a Router?

Question 6: Router Architecture: Hardware vs. Software

A router typically consists of **input ports**, **output ports**, a **switching fabric**, and a **routing processor**.

- a. Which of these are implemented in **hardware** and which in **software**? Why?
- b. Relating to the data plane and control plane, which are implemented in hardware and which are implemented in software? Why?

Solution

The implementation of router components depends on the required processing speed and complexity:

a) Component Implementation:

- **Hardware:** **Input ports, switching fabric, and output ports** are implemented in hardware. This is because their datagram-processing functionality (lookup, switching) must operate at extremely high speeds, far too fast for software-based implementation.
- **Software:** The **routing processor** is implemented in software. It handles complex, less frequent tasks such as executing routing protocols (OSPF, BGP), maintaining routing tables, and communicating with remote controllers (in SDN).

b) Data Plane vs. Control Plane:

- **Data Plane:** Usually implemented in **hardware**. It requires extremely fast processing (nanosecond timescale) to handle the arrival and forwarding of individual packets.
- **Control Plane:** Usually implemented in **software**. It operates at much longer timescales (milliseconds or seconds), managing network-wide processes like path determination, responding to topology changes, and performing management functions.

Question 7: Shadow Copies of Forwarding Tables

Discuss why each **input port** in a high-speed router stores a **shadow copy** of the forwarding table.

Solution

Storing a shadow copy of the forwarding table at each input port is a key design strategy for scalability:

- **Decentralized Lookup:** It allows the forwarding lookup to be performed **locally** at the input port where the packet arrives.
- **Avoiding Bottlenecks:** By processing lookups locally, the router does not need to invoke the centralized routing processor for every arriving packet.
- **High Performance:** This decentralized approach prevents the routing processor from becoming a processing **bottleneck**, which is essential for maintaining the high-speed throughput required in modern routers.

Question 8: Destination-based vs. Generalized Forwarding

What is meant by **destination-based forwarding**? How does this differ from **generalized forwarding**? Which of the two approaches is adopted by **Software-Defined Networking (SDN)**?

Solution

The distinction between these two forwarding paradigms lies in the criteria used for decision-making:

- **Destination-based Forwarding:** Datagrams arriving at a router are forwarded to an output interface based **only** on the final destination IP address of the datagram. This is the traditional approach used by Internet routers.
- **Generalized Forwarding:** Forwarding decisions are made based on **multiple factors** associated with the datagram beyond just the final destination. This include fields from different layers, such as:
 - IP source/destination addresses.
 - TCP/UDP port numbers.
 - Protocol type, etc.
- **SDN Approach:** Software-Defined Networking (SDN) adopts **generalized forwarding**. By using a set of rules (match-action pairs), an SDN router can perform complex operations like load balancing, firewalling, and custom routing policies based on a combination of header fields.

Question 9: Longest Prefix Matching

Suppose that an arriving packet matches two or more entries in a router's forwarding table. With traditional **destination-based forwarding**, what rule does a router apply to determine which of these entries should be used to determine the output port?

Solution

When a packet's destination address matches multiple entries in the forwarding table, the router applies the **Longest Prefix Matching** rule:

- The router identifies all entries that match the destination address.
- It then selects the entry that has the **longest prefix match** with the destination address.
- The packet is switched to the link interface associated with that specific longest-match entry.

Question 10: Types of Switching Fabrics

Three types of **switching fabrics** are used in routers. List and briefly describe each type. Which of these, if any, can send multiple packets across the fabric in **parallel**?

Solution

The three primary types of switching fabrics are:

- 1. Switching via Memory:** The earliest routers were often traditional computers, where switching was performed under the direct control of the CPU. Packets are copied from the input port into system memory and then copied to the appropriate output port. Only one memory read/write can occur at a time, so only one packet can be switched at a time.
- 2. Switching via a Bus:** Input ports transfer packets directly to output ports over a shared bus, without intervention by the routing processor. Although the routing processor is bypassed, the shared nature of the bus means only one packet can cross the bus at any given time.
- 3. Switching via an Interconnection Network:** Uses a more complex network of paths (like a crossbar switch) to connect multiple input ports to multiple output ports.

Parallelism: An **interconnection network** is the only type that can forward **multiple packets in parallel**, provided that they are being forwarded to different output ports. If multiple packets are destined for the same output port, they must still wait and be switched one at a time.

Question 11: Packet Loss at Input Ports

Describe how packet loss can occur at **input ports**. How can packet loss at input ports be eliminated (without using infinite buffers)?

Solution

Packet loss at input ports is primarily caused by internal speed mismatches:

- **Cause of Packet Loss:** If the aggregate rate at which packets arrive at the input ports exceeds the maximum rate at which the switching fabric can transfer them to output ports, packets must be queued at the input ports. If this rate mismatch persists, the input port buffers will eventually overflow, leading to packet loss.
- **Eliminating Packet Loss:** Packet loss at the input ports can be eliminated if the **switching fabric speed** is at least **n times as fast** as the input line speed, where n is the number of input ports.

$$R_{\text{fabric}} \geq n \times R_{\text{line}}$$

In this scenario, even if all n input ports receive packets simultaneously at their maximum line rate, the fabric can switch them all to the output side faster than they can accumulate, thus preventing queue buildup and overflow.

Question 12: Packet Loss at Output Ports

Describe how packet loss can occur at **output ports**. Can this loss be prevented by increasing the **switch fabric speed**?

Solution

Packet loss at output ports occurs due to outgoing line constraints:

- **Cause of Packet Loss:** Even if the input and output line speeds are the same, packet loss can occur if the total rate at which packets arrive from the switching fabric to a **single output port** exceeds the output line speed. If multiple input ports send packets to the same output port simultaneously, a queue forms. If this rate mismatch persists, the output port buffer will eventually overflow, causing packets to be dropped.
- **The Role of Fabric Speed:** Increasing the switch fabric speed **cannot prevent** this problem. In fact, a faster fabric may even accelerate the buildup of queues at the output ports, as it can deliver packets to the output buffer faster than the output line can transmit them.

Question 13: Head-of-the-Line (HOL) Blocking

What is **HOL blocking**? Does it occur in input ports or output ports?

Solution

Head-of-the-Line (HOL) Blocking is a phenomenon where a queued packet is prevented from being switched because it is stuck behind another packet.

- **The Mechanism:** Sometimes a packet that is first in line at an input port queue must wait because there is no available buffer space at the destination output port. When this occurs, **all packets behind it** in the same input queue are also blocked, even if their respective destination output ports are idle and have room to accommodate them.
- **Occurrence:** HOL blocking occurs specifically at the **input ports** of a router.

Question 14: Packet Scheduling Disciplines

We studied **FIFO**, **Priority**, **Round Robin (RR)**, and **Weighted Fair Queueing (WFQ)** packet scheduling disciplines. Which of these queueing disciplines ensure that all packets depart in the order in which they arrived?

Solution

Among the listed disciplines, **only FIFO (First-In, First-Out)** ensures that all packets depart in the exact order in which they arrived.

Rationale for other disciplines:

- **Priority Queueing:** Higher-priority packets can jump ahead of lower-priority packets that arrived earlier.
- **Round Robin (RR) and Weighted Fair Queueing (WFQ):** Packets are divided into classes and served in a rotating or weighted manner. A packet in one class might be served before a packet in another class that arrived slightly earlier.

Question 15: Motivation for Priority Queueing

Give an example showing why a network operator might want one class of packets to be given priority over another class of packets.

Solution

Priority queueing is essential for maintaining the performance and stability of critical network services:

- **Control and Management:** Packets carrying **network management information** (e.g., SNMP traps, routing protocol updates) should receive priority over regular user traffic to ensure the network remains operational and controllable even during congestion.
- **Real-Time Constraints:** Real-time applications, such as **Voice-over-IP (VoIP)** or video conferencing, are extremely sensitive to delay and jitter. These packets must be prioritized over non-real-time traffic (e.g., email or file transfers) to maintain acceptable audio/video quality.

Question 16: Round Robin vs. Weighted Fair Queueing

What is an essential **difference** between **RR** and **WFQ** packet scheduling? Is there a case (Hint: Consider the WFQ weights) where RR and WFQ will behave exactly the same?

Solution

The primary difference lies in how they allocate service capacity among classes:

- **Round Robin (RR):** All service classes are treated **equally**. The scheduler cycles through the classes, serving one packet from each (if available), meaning no class has priority or a larger share of the bandwidth over any other.
- **Weighted Fair Queueing (WFQ):** Service classes are treated **differentially**. Each class i is assigned a weight w_i , and it is guaranteed to receive a fraction of the service capacity equal to $w_i / \sum w_j$ during any interval of time when there are packets to be sent.

Equivalence Case: WFQ and RR behave exactly the same when **all classes in WFQ are assigned the same weight** (e.g., $w_i = 1$ for all i). In this scenario, the proportional share for each class becomes equal, making the scheduling logic identical to Round Robin.

3. 4.3 IP: Internet Protocol

Question 17: The IP Protocol Field

Suppose that Host A sends Host B a TCP segment encapsulated in an IP datagram. When Host B receives the datagram, how does the network layer in Host B know it should pass the segment (the payload of the datagram) to **TCP** rather than to UDP or some other upper-layer protocol?

Solution

The identification of the upper-layer protocol is handled by a specific field in the IP header:

- **The Protocol Field:** The IPv4 datagram header contains an **8-bit protocol field**.
- **Function:** This field contains a specific numerical value that indicates the transport-layer protocol to which the destination host should pass the datagram's payload.
- **Examples:** For example, a value of **6** indicates TCP, while a value of **17** indicates UDP. This acts as a glue between the network and transport layers, similar to how the port number glues the transport and application layers.

Question 18: The IP Time-to-Live (TTL) Field

What field in the **IP header** can be used to ensure that a packet is forwarded through no more than N routers?

Solution

The **Time-to-Live (TTL)** field is used for this purpose:

- **Operation:** The sender initializes the TTL field with a value (e.g., N).
- **Decrementing:** Each router that processes the datagram decrements the TTL value by 1.
- **Discarding:** If the TTL value reaches 0, the router must discard the datagram and typically sends an ICMP "Time Exceeded" message back to the source.
- **Purpose:** This ensures that datagrams do not circulate indefinitely in the network due to routing loops.

Question 19: IP vs. Transport Layer Checksums

Recall that the Internet checksum is used in both transport-layer segments (TCP/UDP headers) and in network-layer datagrams (IP header). Consider a transport-layer segment encapsulated in an IP datagram. Are the checksums in the segment header and datagram header computed over any **common bytes** in the IP datagram? Explain your answer.

Solution

No, they are not computed over any common bytes.

Explanation:

- **IP Header Checksum:** This field is computed **only over the fields in the IP header** itself. Its primary purpose is to ensure the integrity of the routing information (like addresses, TTL, etc.) while the packet moves through the network.
- **Transport Segment Checksum:** This is computed over the **transport-layer segment** (the TCP/UDP header and the actual data payload).
- Since the IP header and the transport segment are discrete parts of the overall datagram, their checksums do not overlap.

Question 20: IP Fragmentation and Reassembly

When a large datagram is fragmented into multiple smaller datagrams, where are these smaller datagrams **reassembled** into a single larger datagram?

Solution

The reassembly of the fragments of an IP datagram is done exclusively in the **destination host**.

Why in the host and not routers?

- **Simplicity:** Doing reassembly only at the end-systems keeps the network core (routers) simple and fast.
- **Efficiency:** Routers don't need to buffer fragments or wait for all parts of a datagram to arrive before forwarding them, which reduces potential bottlenecks.
- **Fate Sharing:** If a fragment is lost, the destination host can simply time out and let the upper layers handle retransmission.

Question 21: Router IP Addresses

Do routers have IP addresses? If so, how many?

Solution

Yes, routers have IP addresses.

A router typically has **one IP address for each interface** (physically or logically connectivity point between the router and one of its links). Since a router is designed to forward packets between different networks, it must have a presence (an interface and an associated IP address) on each of the networks it connects.

Question 22: IP Address Binary Conversion

What is the 32-bit binary equivalent of the IP address **223.1.3.27**?

Solution

To convert **223.1.3.27** to binary, we convert each decimal octet individually:

- **223:** $128 + 64 + 16 + 8 + 4 + 2 + 1 \rightarrow 11011111$
- **1:** $1 \rightarrow 00000001$
- **3:** $2 + 1 \rightarrow 00000011$
- **27:** $16 + 8 + 2 + 1 \rightarrow 00011011$

Full 32-bit binary equivalent:

11011111 00000001 00000011 00011011

Question 23: DHCP Configuration Values

Visit a host that uses DHCP to obtain its IP address, network mask, default router, and IP address of its local DNS server. List these values.

Solution

While these values vary depending on the specific network, a typical example for a host on a home or office network would be:

- **IP Address:** 192.168.1.105
- **Network Mask:** 255.255.255.0
- **Default Router (Gateway):** 192.168.1.1
- **Local DNS Server:** 192.168.1.1 (often the router) or 8.8.8.8 (Google Public DNS)

How to find these values:

- **Windows:** Open Command Prompt and type `ipconfig /all`.
- **Linux/macOS:** Open Terminal and type `ifconfig` or `nmcli dev show` (or check `/etc/resolv.conf` for DNS).

Question 24: Interfaces and Forwarding Tables in a Path

Suppose there are **three routers** between a source host and a destination host. Ignoring fragmentation, an IP datagram sent from the source host to the destination host will travel over how many **interfaces**? How many **forwarding tables** will be indexed to move the datagram from the source to the destination?

Solution

For a path with **three routers**:

- **Number of Interfaces: 8**
 - Source Host: 1 interface.
 - Router 1: 2 interfaces (inbound and outbound).
 - Router 2: 2 interfaces (inbound and outbound).
 - Router 3: 2 interfaces (inbound and outbound).
 - Destination Host: 1 interface.
 - Total: $1 + 2 + 2 + 2 + 1 = 8$ interfaces.
- **Forwarding Tables Indexed: 3**
 - Forwarding decisions based on destination IP addresses are made by the **routers** along the path.
 - Each of the 3 routers will index its forwarding table to determine the appropriate output interface for the datagram.

Question 25: Datagram Overhead Calculation

Suppose an application generates chunks of **40 bytes** of data every 20 msec, and each chunk gets encapsulated in a **TCP segment** and then an **IP datagram**. What percentage of each datagram will be **overhead**, and what percentage will be **application data**?

Solution

To calculate the percentages, we first determine the sizes of the headers and the payload:

1. **Application Data:** 40 bytes.
2. **TCP Header:** 20 bytes (standard size without options).
3. **IP Header:** 20 bytes (standard size without options).

Calculations:

- **Total Overhead:** 20 (TCP) + 20 (IP) = **40** bytes.
- **Total Datagram Size:** 40 (Data) + 40 (Overhead) = **80** bytes.

Percentages:

- **Percentage Overhead:** $\frac{40}{80} \times 100 = 50\%$.
- **Percentage Application Data:** $\frac{40}{80} \times 100 = 50\%$.

Question 26: NAT and Home Networking

Suppose you purchase a wireless router and connect it to your cable modem. Also suppose that your ISP dynamically assigns your connected device (the router) one IP address. You have five PCs at home that wirelessly connect to your wireless router. How are IP addresses assigned to the five PCs? Does the wireless router use **NAT**? Why or why not?

Solution

In this scenario, the network operates as follows:

- **IP Assignment:** Typically, the wireless router includes a built-in **DHCP server**. This server is used to dynamically assign local (private) IP addresses to the five PCs and to manage the internal router interface.
- **Use of NAT:** **Yes**, the wireless router uses **Network Address Translation (NAT)**.
- **Reasoning:** The router only obtains **one public IP address** from the ISP. Since there are multiple devices (the five PCs) that need to communicate with the Internet simultaneously, the router must use NAT to map their multiple private IP addresses to that single public address using different port numbers.

Question 27: Route Aggregation

What is meant by the term “**route aggregation**”? Why is it useful for a router to perform route aggregation?

Solution

Route aggregation (also known as route summarization) occurs when an ISP or a router uses a **single prefix** to advertise the reachability of multiple, more specific networks.

Utility:

- **Efficiency:** It allows an ISP to advertise a single summary prefix to the rest of the Internet instead of individual prefixes for every customer network it manages.
- **Scalability:** This significantly reduces the size of the global routing table (RIB/FIB), as core routers only need to store one entry for a large block of addresses rather than hundreds of smaller ones.
- **Stability:** It hides "flapping" (frequent status changes) of internal networks from the rest of the Internet, as the summary route remains valid as long as at least one part of the aggregated range is reachable.

Question 28: Plug-and-Play or Zeroconf Protocols

What is meant by a "**plug-and-play**" or "**zeroconf**" protocol?

Solution

A **plug-and-play** or **zeroconf** (zero-configuration) protocol is one that is able to **automatically configure** a host's network-related aspects (such as IP address, subnet mask, default gateway, and DNS server) to connect the host to a network without manual intervention by a user or administrator.

Example:

- **DHCP (Dynamic Host Configuration Protocol):** This is the most common plug-and-play protocol used in IP networks. When a device joins a network, it automatically broadcasts a request for configuration, and a DHCP server provides all the necessary information for it to start communicating immediately.

Question 29: Private Network Addresses

What is a **private network address**? Should a datagram with a private network address ever be present in the larger **public Internet**? Explain.

Solution

A **private network address** is an IP address reserved for use within internal networks. These addresses are only meaningful to devices **within that specific network** and are not globally unique.

Presence in Public Internet: No, a datagram with a private network address should never be present in the larger public Internet.

Explanation:

- **Non-Uniqueness:** Private addresses (like 192.168.x.x) are reused by millions of private networks worldwide. If a packet with such an address entered the public Internet, routers would not know which specific private network it belonged to, leading to ambiguity and routing failures.
- **RFC 1918:** Standards (specifically RFC 1918) designate these ranges specifically for private use. Public routers are configured to **drop** any packets with private source or destination addresses.
- **NAT Necessity:** For devices with private addresses to communicate with the Internet, a **NAT (Network Address Translation)** device must translate the private address into a globally unique public IP address.

Question 30: Comparing IPv4 and IPv6 Header Fields

Compare and contrast the **IPv4** and the **IPv6** header fields. Do they have any fields in common?

Solution

IPv6 was designed to streamline packet processing and handle the exhaustion of IPv4 addresses.

Key Systemic Differences:

- **Fixed Length:** The IPv6 header has a fixed length of **40 bytes**, whereas the IPv4 header is variable (min. 20 bytes).
- **Address Size:** IPv6 uses **128-bit** addresses, compared to the **32-bit** addresses in IPv4.
- **Checksum:** IPv6 eliminates the header checksum to reduce processing time at routers.

Analogous Fields (“Similar in Spirit”): While many fields were renamed or modified, several are functionally similar:

IPv4 Field	Analogous IPv6 Field
Type of Service (ToS)	Traffic Class
Datagram Length	Payload Length
Protocol	Next Header
Time to Live (TTL)	Hop Limit

Common Fields: Strictly speaking, **only the Version field** is common to both headers in name and position, though its value is obviously different (4 vs 6).

Question 31: IPv6 Tunneling through IPv4

It has been said that when IPv6 tunnels through IPv4 routers, IPv6 treats the IPv4 tunnels as **link-layer protocols**. Do you agree with this statement? Why or why not?

Solution

Yes, I agree with this statement.

Reasoning: In tunneling, the **entire IPv6 datagram** (including its header and payload) is encapsulated within the payload field of an **IPv4 datagram**.

From the perspective of the IPv6 nodes at the ends of the tunnel:

- The tunnel acts simply as a point-to-point "link" that carries the IPv6 packet from one IPv6 router to another.
- The fact that this "link" actually consists of multiple IPv4 routers and path segments is transparent to the IPv6 layer, just like the internal workings of an Ethernet switch or a physical wire are transparent to the network layer.

4.4 Generalized Forwarding and SDN

Question 32: Generalized vs. Destination-Based Forwarding

How does **generalized forwarding** differ from **destination-based forwarding**?

Solution

Forwarding consists of two main operations: **match** and **action**. The primary differences lie in the flexibility of these operations:

- **Destination-Based Forwarding:**
 - **Match:** The router looks up **only** the destination IP address in its forwarding table.
 - **Action:** The router sends the packet through the switching fabric to a specific output port.
- **Generalized Forwarding (SDN/OpenFlow):**
 - **Match:** The match can be made over **multiple header fields** across different layers (L2 Mac addresses, L3 IP addresses, L4 Port numbers, etc.).
 - **Action:** The action is significantly more flexible and can include:
 - * Forwarding to one or more ports.
 - * **Load-balancing** across multiple interfaces.
 - * **Rewriting** header values (e.g., NAT or MAC address changes).
 - * **Dropping** packets (firewalling).
 - * Sending a packet to a controller or special server for further processing.

Question 33: Forwarding Table vs. OpenFlow Flow Table

What is the difference between a **forwarding table** encountered in destination-based forwarding (Sec 4.1) and OpenFlow's **flow table** (Sec 4.4)?

Solution

The primary difference lies in the complexity and information contained in each entry:

- **Destination-Based Forwarding Table Entry:**
 - Consists of an **IP destination prefix** (used for matching).
 - Consists of the **outgoing link interface** (the action associated with the match).
 - Focuses strictly on the destination L3 address.
- **OpenFlow Flow Table Entry:**
 - **Match Fields:** A set of header field values (L2, L3, and L4) to which the incoming packet is compared.
 - **Counters:** A set of statistics (e.g., number of packets/bytes matched) that are updated as packets process through the table.
 - **Actions:** A set of instructions to be taken for matched packets (forwarding, dropping, modifying, etc.).

Question 34: The “Match Plus Action” Paradigm

What is meant by the “**match plus action**” operation of a router or switch? In the case of destination-based forwarding packet switch, what is matched and what is the action taken? In the case of an **SDN**, name three fields that can be matched, and three actions that can be taken.

Solution

“**Match plus action**” means that a router or switch first attempts to find a **match** between various header values of an incoming packet and entries in a flow table (or forwarding table). Once a match is found, the device performs a specific **action** (or set of actions) associated with that entry.

1. Destination-Based Forwarding:

- **Matched:** Only the **Destination IP Address** of the arriving packet.
- **Action:** Deciding the **output interface(s)** to which the packet will be forwarded.

2. Software-Defined Networking (SDN):

- **Example Fields Matched:**
 - Source IP Address.
 - TCP/UDP Source Port.
 - Source MAC Address.
- **Example Actions Taken:**
 - **Forwarding** the packet to specific output ports.
 - **Dropping** the packet (discarding it).
 - **Modifying** header fields (e.g., rewriting IP or MAC addresses).

Question 35: Matchable and Non-Matchable Fields in OpenFlow 1.0

Name three header fields in an IP datagram that can be “**matched**” in **OpenFlow 1.0** generalized forwarding. What are three IP datagram header fields that **cannot** be “matched” in OpenFlow?

Solution

In OpenFlow 1.0, the match operation is restricted to specific fields in the Ethernet (L2), IP (L3), and TCP/UDP (L4) headers.

1. Matchable Fields (Examples):

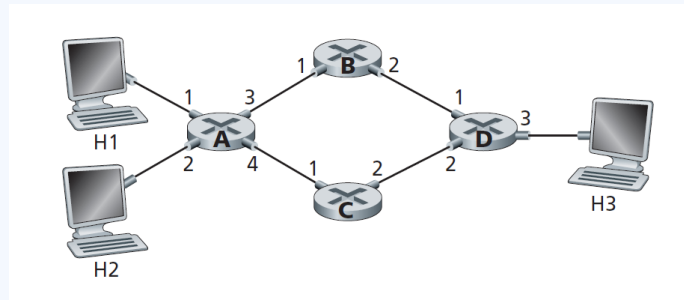
- **IP Source Address** (L3).
- **TCP Source Port** (L4).
- **Source MAC Address** (L2 - though encapsulated in Ethernet, it is part of the “flow” definition).

2. Non-Matchable Fields (Examples):

- **TTL (Time-to-Live):** OpenFlow 1.0 cannot match based on the hop count.
- **Datagram Length:** Matches cannot be made based on the size of the packet.
- **Header Checksum:** Since this field changes whenever the TTL changes, it is not a viable match field for flow entries.

Question 36: Destination-Based Forwarding Limitations

Consider the network below:



- Show the forwarding table in **Router A**, such that all traffic destined to host **H3** is forwarded through **interface 3**.
- Can you write down a forwarding table in Router A, such that all traffic from **H1** destined to host **H3** is forwarded through **interface 3**, while all traffic from **H2** destined to host **H3** is forwarded through **interface 4**? (Hint: This is a trick question.)

Solution

Part a: Forwarding Table in Router A

Destination Address	Link Interface
H3	3

Part b: Feasibility No, this is not possible with traditional destination-based forwarding.

Explanation: In destination-based forwarding, the match operation is performed **solely** using the destination IP address of the datagram. Since both H1 and H2 are sending traffic to the **same destination (H3)**, Router A will observe the same destination address in both types of traffic. It cannot differentiate between the two flows based on their source. To achieve the specific routing requested in Part b, **generalized forwarding** would be required to match on the source address as well.

Question 37: Switching Fabric Concurrent Access

Suppose two packets arrive to two different input ports of a router at exactly the same time. Also suppose there are no other packets anywhere in the router.

- a. Suppose the two packets are to be forwarded to two different output ports. Is it possible to forward the two packets through the switch fabric at the same time when the fabric uses a **shared bus**?
- b. Suppose the two packets are to be forwarded to two different output ports. Is it possible to forward the two packets through the switch fabric at the same time when the fabric uses **switching via memory**?
- c. Suppose the two packets are to be forwarded to the **same output port**. Is it possible to forward the two packets through the switch fabric at the same time when the fabric uses a **crossbar**?

Solution

- a. **No.** In a shared bus architecture, only one packet can be transmitted over the bus at any given time. Even if they are heading to different output ports, they must take turns using the single bus.
- b. **No.** Switching via memory typically involves copying the packet from the input port to the system memory and then to the output port. This usually happens over a shared system bus, which allows only one memory read/write operation at a time.
- c. **No.** Although a crossbar (interconnection network) allows multiple packets to be forwarded in parallel to **different** output ports, it cannot send two packets to the **same** output port simultaneously. The output bus/port itself is a serial resource.

Question 38: Maximum Input Queuing Delay

It was said that if R_{switch} is N times faster than R_{line} , then only negligible queuing will occur at the input ports, even if all the packets are to be forwarded to the same output port. Now suppose that $R_{switch} = R_{line}$, but all packets are to be forwarded to **different output ports**. Let D be the time to transmit a packet. As a function of D , what is the maximum input queuing delay for a packet for the (a) memory, (b) bus, and (c) crossbar switching fabrics?

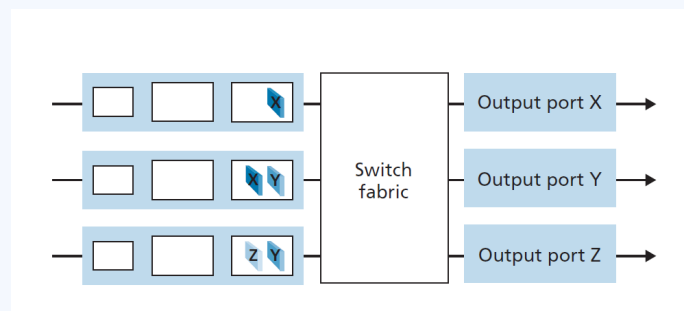
Solution

Under the condition $R_{switch} = R_{line}$ and n input ports:

- Memory:** $(n - 1)D$. Since only one packet can be processed at a time through the shared bus to memory, the n -th packet must wait for the previous $n - 1$ packets to be processed.
- Bus:** $(n - 1)D$. Similarly, only one packet can use the bus at a time. If n packets arrive at once, the last one waits until the others have finished using the bus.
- Crossbar:** **0**. A crossbar can forward multiple packets in parallel as long as they are destined for different output ports. Since $R_{switch} = R_{line}$ for each path and all destinations are different, all packets can be forwarded simultaneously without delay.

Question 39: Switching Fabric Scheduling

Consider the switch shown below. Suppose that all datagrams have the same fixed length, that the switch operates in a slotted, synchronous manner, and that in one time slot a datagram can be transferred from an input port to an output port.



The switch fabric is a **crossbar** so that at most one datagram can be transferred to a given output port in a time slot, but different output ports can receive datagrams from different input ports in a single time slot.

- What is the **minimal** number of time slots needed to transfer the packets shown from input ports to their output ports, assuming any input queue scheduling order you want (i.e., it need not have HOL blocking)?
- What is the **largest** number of slots needed, assuming the worst-case scheduling order you can devise, assuming that a non-empty input queue is never idle?

Note: Actually, if the first datagram in the bottom input queue is X, then the worst case would require 4 or more time slots depending on the conflicts.

Solution

Minimal Time Slots: 3 One possible optimal scheduling is:

- **Slot 1:** Send X from the top input queue and Y from the middle input queue.
- **Slot 2:** Send X from the middle input queue and Y from the bottom input queue.
- **Slot 3:** Send Z from the bottom input queue.

Maximal Time Slots: 3 (typically) Based on the assumption that a non-empty input queue is **never idle**, the first time slot will always attempt to send packets to available outputs.

- If the first datagram in the bottom queue is X, it conflicts with the top queue's X. If we prioritize the top queue, the bottom queue stays idle (or is blocked by HOL).
- Even in worst-case scenarios with the provided constraints, 3 slots are usually sufficient unless Head-of-the-Line (HOL) blocking specifically delays a unique final packet. If the bottom queue starts with X, and we always prioritize other queues, those 3 packets in the bottom queue might take 3 full slots after others are finished, potentially totaling 4 slots.

Question 40: Weighted Fair Queueing (WFQ) Sequences

Suppose that the **WFQ** scheduling policy is applied to a buffer that supports three classes, and suppose the weights are **0.5, 0.25, and 0.25** for the three classes.

- a. Suppose that each class has a large number of packets in the buffer. In what sequence might the three classes be served in order to achieve the WFQ weights?
- b. Suppose that classes 1 and 2 have a large number of packets in the buffer, and there are no class 3 packets in the buffer. In what sequence might the two classes be served in to achieve the WFQ weights?

Solution

The weights 0.5, 0.25, and 0.25 correspond to a ratio of **2 : 1 : 1**.

Part a: All classes have packets To achieve the 2:1:1 ratio, Class 1 must be served twice as often as Class 2 or Class 3. A natural repeating sequence would be:

1, 1, 2, 3, 1, 1, 2, 3, ...

In every cycle of 4 packets, Class 1 gets 2 slots (50%), Class 2 gets 1 slot (25%

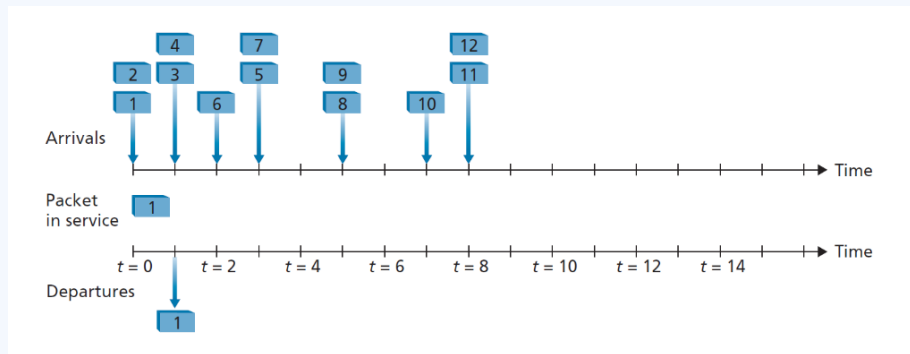
Part b: Only classes 1 and 2 have packets When Class 3 is empty, the scheduler distributes the service between the remaining active classes proportional to their original relative weights. The ratio of Class 1 to Class 2 remains **2 : 1**. A natural repeating sequence would be:

1, 1, 2, 1, 1, 2, ...

In every 3 packets, Class 1 gets 2/3 of the service and Class 2 gets 1/3, maintaining the correct relative weights.

Question 41: Comparing Scheduling Policies

Consider the packet arrival and service timing diagram below:



- Assuming **FIFO** service, indicate the time at which packets 2 through 12 each leave the queue. For each packet, what is the delay between its arrival and the beginning of the slot in which it is transmitted? What is the average of this delay over all 12 packets?
- Now assume a **priority service**, and assume that odd-numbered packets are high priority, and even-numbered packets are low priority. Indicate the time at which packets 2 through 12 each leave the queue. For each packet, what is the delay between its arrival and the beginning of the slot in which it is transmitted? What is the average of this delay over all 12 packets?
- Now assume **round robin** service. Assume that packets 1, 2, 3, 6, 11, and 12 are from class 1, and packets 4, 5, 7, 8, 9, and 10 are from class 2. Indicate the time at which packets 2 through 12 each leave the queue. For each packet, what is the delay between its arrival and its departure? What is the average delay over all 12 packets?
- Now assume **weighted fair queueing (WFQ)** service. Assume that odd-numbered packets are from class 1, and even-numbered packets are from class 2. Class 1 has a WFQ weight of 2, while class 2 has a WFQ weight of 1. Note that it may not be possible to achieve an idealized WFQ schedule as described in the text, so indicate why you have chosen the particular packet to go into service at each time slot. For each packet what is the delay between its arrival and its departure? What is the average delay over all 12 packets?
- What do you notice about the average delay in all four cases (FIFO, RR, priority, and WFQ)?

Solution

Part a: FIFO Service

Packet Index	Leave Time	Delay
2	2	2
3	3	2
4	4	3
5	6	3
6	5	3
7	7	4
8	8	3
9	9	4
10	10	3
11	11	3
12	12	4

Average Delay: 3.091

Part b: Priority Service (Odd: High, Even: Low)

Index	Leave	Delay	Index	Leave	Delay
2	3	3	8	10	5
3	2	1	9	6	1
4	7	6	10	11	4
5	4	1	11	9	1
6	8	6	12	12	4
7	5	2			

Average Delay: 3.091

Part c: Round Robin Service (Class 1: 1,2,3,6,11,12; Class 2: 4,5,7,8,9,10)

Index	Leave	Delay	Index	Leave	Delay
2	3	3	8	8	3
3	5	4	9	10	5
4	2	1	10	12	5
5	4	1	11	9	1
6	7	5	12	11	3
7	6	3			

Average Delay: 3.091

Part d: WFQ Service (Class 1: Odd, W=2; Class 2: Even, W=1)

Index	Leave	Delay	Index	Leave	Delay
2	3	3	8	10	5
3	2	1	9	7	2
4	6	5	10	11	4
5	4	1	11	9	1
6	8	6	12	12	4
7	5	2			

Average Delay: 3.091

Part e: Observation All average delays remain the same (3.091), no matter what scheduling algorithm is used. The choice of algorithm affects individual packet delay but not the system-wide average for a fixed set of arrivals and service rates.

Question 42: Forwarding Exercise

- a. Indicate the leave times for a specific traffic scenario (Part a-c).

Solution

- a. They leave at time 5, 6, 2, 4, 3, 7, 8, 10, 11, 9, 12 respectively.
- b. They leave at time 2, 4, 3, 7, 5, 6, 8, 10, 11, 9, 12 respectively.
- c. They leave at time 2, 3, 4, 10, 7, 5, 6, 8, 9, 12, 11 respectively.

Question 43: Longest Prefix Matching Forwarding Table

Consider a datagram network using 32-bit host addresses. Suppose a router has four links, numbered 0 through 3, and packets are to be forwarded to the link interfaces as follows:

Destination Address Range	Link Interface
11100000 00000000 00000000 00000000 through 11100000 00111111 11111111 11111111	0
11100000 01000000 00000000 00000000 through 11100000 01000000 11111111 11111111	1
11100000 01000001 00000000 00000000 through 11100001 01111111 11111111 11111111	2
Otherwise	3

- a. Provide a forwarding table that has five entries, uses **longest prefix matching**, and forwards packets to the correct link interfaces.
- b. Describe how your forwarding table determines the appropriate link interface for datagrams with destination addresses:
- 11001000 10010001 01010001 01010101
 - 11100001 01000000 11000011 00111100
 - 11100001 10000000 00010001 01110111

Solution

Part a: Forwarding Table (LPM)

Prefix Match	Link Interface
11100000 00	0
11100000 01000000	1
1110000	2
11100001 1	3
Otherwise	3

Part b: Interface Determination

- **11001000...**: This address does not match any of the first four prefixes. It matches the **Otherwise** entry, resulting in **Link Interface 3**.
- **11100001 01000000...**: This address matches the prefix 1110000 (7 bits). It does not match prefix 4 as its 9th bit is 0. Longest match is entry 3, resulting in **Link Interface 2**.
- **11100001 10000000...**: This address matches both 1110000 (7 bits) and 11100001 1 (9 bits). Using longest prefix matching, it matches entry 4, resulting in **Link Interface 3**.

Question 44: Address Range Capacity

Consider a datagram network using 32-bit addresses and the following range matching rules:

Destination Address Range (First Byte)			Link Interface
00000000	through	00111111	0
01000000	through	01011111	1
01100000	through	01111111	2
10000000	through	10111111	2
11000000	through	11111111	3

What is the total number of addresses assigned to each interface?

Solution

Each range covers a set of prefixes (represented by the first byte). Since the addresses are 32-bit, each 8-bit prefix represents $2^{32-8} = 2^{24}$ addresses.

- **Interface 0:** Range 0-63 (Size 64).
Number of addresses = $64 \times 2^{24} = 2^6 \times 2^{24} = 2^{30} = \mathbf{1,073,741,824}$.
- **Interface 1:** Range 64-95 (Size 32).
Number of addresses = $32 \times 2^{24} = 2^5 \times 2^{24} = 2^{29} = \mathbf{536,870,912}$.
- **Interface 2:** Ranges 96-127 (Size 32) + 128-191 (Size 64). Total Size = 96.
Number of addresses = $96 \times 2^{24} = 1.5 \times 2^6 \times 2^{24} = \mathbf{1,610,612,736}$.
- **Interface 3:** Range 192-255 (Size 64).
Number of addresses = $64 \times 2^{24} = 2^{30} = \mathbf{1,073,741,824}$.

Total Check: $\Sigma = (64 + 32 + 96 + 64) \times 2^{24} = 256 \times 2^{24} = 2^8 \times 2^{24} = 2^{32}$, which covers the entire address space.

Question 45: 8-bit LPM Range Mapping

Consider a datagram network using 8-bit host addresses. Suppose a router uses **longest prefix matching** and has the following forwarding table:

Prefix Match	Interface
1	0
10	1
111	2
Otherwise	3

For each of the four interfaces, give the associated range of destination host addresses and the number of addresses in the range.

Solution

To determine the range for each interface, we apply the longest prefix matching rule to all $2^8 = 256$ possible addresses (00000000 to 11111111).

- **Interface 1 (Prefix 10):** Any address starting with 10 matches both prefix 1 (length 1) and prefix 10 (length 2). Longest match is Interface 1. Range: 10000000 to 10111111. **Total: 64 addresses.**
- **Interface 2 (Prefix 111):** Any address starting with 111 matches prefix 1 (length 1) and prefix 111 (length 3). Longest match is Interface 2. Range: 11100000 to 11111111. **Total: 32 addresses.**
- **Interface 0 (Prefix 1):** This interface handles addresses that match prefix 1 but **not** longer prefixes (10 or 111). This leaves addresses starting with 110. Range: 11000000 to 11011111. **Total: 32 addresses.**
- **Interface 3 (Otherwise):** Handles all addresses starting with 0. Range: 00000000 to 01111111. **Total: 128 addresses.**

Summary Table:

Interface	Address Range	Count
0	11000000 - 11011111	32
1	10000000 - 10111111	64
2	11100000 - 11111111	32
3	00000000 - 01111111	128

Question 46: Subnet Addressing Constraints

Consider a router that interconnects three subnets: **Subnet 1**, **Subnet 2**, and **Subnet 3**. Suppose all interfaces in each of these three subnets are required to have the prefix 223.1.17/24.

Suppose the requirements are:

- **Subnet 1:** At least 60 interfaces.
- **Subnet 2:** At least 90 interfaces.
- **Subnet 3:** At least 12 interfaces.

Provide three network addresses (of the form **a.b.c.d/x**) that satisfy these constraints.

Solution

To satisfy the requirements, we must choose subnet masks that provide enough host addresses for each subnet.

- **Subnet 2 (90 interfaces):** To support 90 interfaces, we need at least 7 host bits ($2^7 = 128$). This gives a **/25** mask. Address: **223.1.17.128/25** (Range: .128 to .255, capacity: 126 hosts).
- **Subnet 1 (60 interfaces):** To support 60 interfaces, we need at least 6 host bits ($2^6 = 64$). This gives a **/26** mask. Address: **223.1.17.0/26** (Range: .0 to .63, capacity: 62 hosts).
- **Subnet 3 (12 interfaces):** To support 12 interfaces, we need at least 4 host bits ($2^4 = 16$). This gives a **/28** mask. Address: **223.1.17.64/28** (Range: .64 to .79, capacity: 14 hosts).

Note: The initial user-provided address for Subnet 3 was 223.1.17.192/28, which would overlap with Subnet 2's range. The above allocation ensures all subnets are distinct and non-overlapping within the 223.1.17/24 space.

Question 47: CIDR Notation Conversion

Rewrite the forwarding table from Question 43 using the **a.b.c.d/x** (CIDR) notation instead of the binary string notation.

Solution

Forwarding Table (CIDR Notation)

Prefix Match	Link Interface
224.0.0.0/10	0
224.64.0.0/16	1
224.0.0.0/7	2
225.128.0.0/9	3
Otherwise	3

Question 48: Subnet IP Address Assignment

Consider a subnet with prefix **128.119.40.128/26**. Give an example of one IP address (of the form **xxx.xxx.xxx.xxx**) that can be assigned to this network.

Solution

A **/26** mask means the subnet has $2^{32-26} = 2^6 = 64$ addresses. The range for **128.119.40.128/26** is from **128.119.40.128** to **128.119.40.191**.

Any IP address in this range (excluding the network and broadcast addresses for specific host assignments) can be used. Examples include:

- 128.119.40.129
- 128.119.40.130
- 128.119.40.190

Question 49: ISP Address Block Partitioning

Suppose an ISP owns the block of addresses of the form **128.119.40.64/26**. Suppose it wants to create four subnets from this block, with each block having the same number of IP addresses. What are the prefixes (of form **a.b.c.d/x**) for the four subnets?

Solution

The original block **128.119.40.64/26** has 64 addresses (2^6). To create four equal-sized subnets, each subnet must have:

$$64/4 = 16 \text{ addresses per subnet}$$

A block with 16 addresses (2^4) requires a mask of $32 - 4 =$ **/28**.

The four subnets starting from .64 are:

1. **128.119.40.64/28** (Range: .64 to .79)
2. **128.119.40.80/28** (Range: .80 to .95)
3. **128.119.40.96/28** (Range: .96 to .111)
4. **128.119.40.112/28** (Range: .112 to .127)

Question 50: Datagram Count Calculation

Suppose datagrams are limited to **1,500 bytes** (including header) between source Host A and destination Host B. Assuming a **20-byte IP header**, how many datagrams would be required to send an MP3 consisting of **5 million bytes**? Explain how you computed your answer.

Solution

To compute the number of datagrams, we must account for the overhead of both the network and transport layers.

Assumptions:

- IP Header: 20 bytes.
- TCP Header: 20 bytes (logical assumption for data transfer like an MP3 file).
- MTU (Maximum Transmission Unit): 1,500 bytes.

Calculation:

1. **Payload per Datagram:** Each datagram can carry a total of 1500 bytes. Subtracting the IP and TCP headers ($1500 - 40$), the actual MP3 data payload per datagram is **1,460 bytes**.
2. **Total Datagrams:** Divide the total file size by the payload per datagram:

$$\frac{5,000,000}{1,460} \approx 3424.65$$

Since we cannot send a fraction of a datagram, we need **3,425 datagrams**.

3. Datagram Distribution:

- The first **3,424 datagrams** will be full-sized (1,500 bytes).
- The last datagram will carry the remaining data.

$$5,000,000 - (3424 \times 1460) = 5,000,000 - 4,999,040 = 960 \text{ bytes (payload)}$$

$$\text{Total size of the last datagram} = 960 + 40 = \mathbf{1000 \text{ bytes.}}$$

Note: In this scenario, there is no fragmentation because the source host creates datagrams that already fit within the link MTU.

Question 51: NAT Addressing and Translation Table

Consider a network setup where an ISP assigns the router a public IP address **24.34.112.235**. The internal home network uses the address block **192.168.1/24**.

- a. Assign addresses to all interfaces in the home network (assuming three hosts and one router interface).
- b. Suppose each host has two ongoing TCP connections, all to port 80 at host 128.119.40.86. Provide the six corresponding entries in the NAT translation table.

Solution

Part a: Home network addresses A possible assignment for the internal interfaces is:

- **Router (Internal Interface):** 192.168.1.4
- **Host 1:** 192.168.1.1
- **Host 2:** 192.168.1.2
- **Host 3:** 192.168.1.3

Part b: NAT Translation Table The table maps the internal (LAN side) private IP and port to the external (WAN side) public IP and a unique port.

WAN Side	LAN Side
24.34.112.235, 4000	192.168.1.1, 3345
24.34.112.235, 4001	192.168.1.1, 3346
24.34.112.235, 4002	192.168.1.2, 3445
24.34.112.235, 4003	192.168.1.2, 3446
24.34.112.235, 4004	192.168.1.3, 3545
24.34.112.235, 4005	192.168.1.3, 3546

Note: The WAN ports (4000-4005) are uniquely assigned by the router to differentiate the six connections.

Question 52: Detecting Hosts behind NAT

Suppose you are interested in detecting the number of hosts behind a NAT. You observe that the IP layer stamps an identification number (IPID) sequentially on each IP packet. The ID of the first packet generated by a host is random, and subsequent IDs are sequentially assigned.

- a. Based on this observation, and assuming you can sniff all packets sent by the NAT to the outside, outline a simple technique to detect the number of unique hosts behind the NAT. Justify your answer.
- b. If the identification numbers are randomly assigned instead of sequentially, would your technique work? Justify your answer.

Solution

Part a: Cluster-based Detection Since each host generates a sequence of IP packets with sequential IDs starting from a random point, we can group captured packets into **clusters (or chains)** based on consecutive ID values.

- Because the initial IDs are chosen from a large 16-bit space ($0 - 65535$), it is highly likely that different hosts' ID sequences will start far apart.
- By counting the number of distinct ID clusters/chains identified in the captured traffic, we can estimate the number of hosts behind the NAT.

For practical implementation, see: "A Technique for Counting NATted Hosts" by Steven M. Bellovin (IMW'02).

Part b: Random Assignment No, the technique would **not work**. If IDs are randomly assigned for every packet, there will be no sequential relationship between packets from the same host. In the sniffed data, we would observe a random distribution of IDs that cannot be grouped into meaningful clusters, making it impossible to differentiate between hosts.

Question 53: Impact of NATs on P2P Applications

In this problem, we explore the impact of NATs on P2P applications. Suppose a peer with username **Arnold** discovers through querying that a peer with username **Bernard** has a file it wants to download. Suppose that Bernard and Arnold are both **behind a NAT**.

Try to devise a technique that will allow Arnold to establish a **TCP connection** with Bernard without application-specific NAT configuration. If you have difficulty devising such a technique, discuss why.

Solution

Without specialized configuration (like UPnP or a relay server), it is **not possible** to establish a direct TCP connection between Arnold and Bernard in a standard environment.

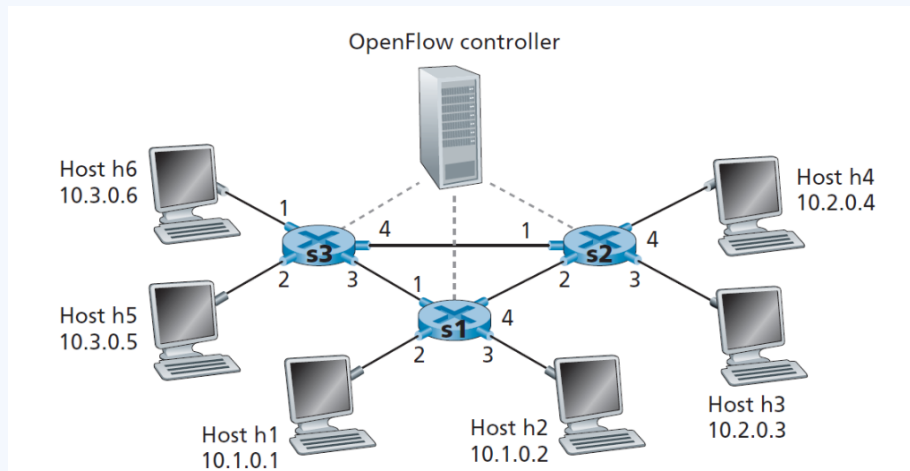
Justification:

- To establish a direct TCP connection, one peer must initiate the connection by sending a **SYN** packet to the other.
- However, NATs are typically configured to drop any unsolicited **SYN** packets arriving from the WAN (outside) side that do not correspond to an entry in the translation table created by an internal host.
- Since both peers are behind NATs, if Arnold tries to connect to Bernard, Bernard's NAT will drop the SYN. Similarly, if Bernard tries to connect to Arnold, Arnold's NAT will drop the packet.

Thus, neither host can successfully initiate the three-way handshake required for a TCP connection.

Question 54: SDN OpenFlow Forwarding behavior

Consider the SDN OpenFlow network shown in Figure 4.30.



Suppose that the desired forwarding behavior for datagrams arriving at s2 is as follows:

- any datagrams arriving on input port 1 from hosts h5 or h6 that are destined to hosts h1 or h2 should be forwarded over output port 2;
- any datagrams arriving on input port 2 from hosts h1 or h2 that are destined to hosts h5 or h6 should be forwarded over output port 1;
- any arriving datagrams on input ports 1 or 2 and destined to hosts h3 or h4 should be delivered to the host specified;
- hosts h3 and h4 should be able to send datagrams to each other.

Specify the flow table entries in s2 that implement this forwarding behavior.

Solution

S2 Flow Table

Match	Action
Ingress Port = 1; IP Src = 10.3.*.*; IP Dst = 10.1.*.*	Forward (2)
Ingress Port = 2; IP Src = 10.1.*.*; IP Dst = 10.3.*.*	Forward (1)
Ingress Port = 1; IP Dst = 10.2.0.3	Forward (3)
Ingress Port = 2; IP Dst = 10.2.0.3	Forward (3)
Ingress Port = 1; IP Dst = 10.2.0.4	Forward (4)
Ingress Port = 2; IP Dst = 10.2.0.4	Forward (4)
Ingress Port = 4	Forward (3)
Ingress Port = 3	Forward (4)

Question 55: SDN OpenFlow Forwarding behavior (Clockwise/Counter-clockwise)

Consider again the SDN OpenFlow network shown in Figure 4.30.

Suppose that the desired forwarding behavior for datagrams arriving from hosts h3 or h4 at s2 is as follows:

- any datagrams arriving from host h3 and destined for h1, h2, h5 or h6 should be forwarded in a **clockwise** direction in the network;
- any datagrams arriving from host h4 and destined for h1, h2, h5 or h6 should be forwarded in a **counter-clockwise** direction in the network.

Specify the flow table entries in s2 that implement this forwarding behavior.

Solution

S2 Flow Table

Match	Action
Ingress Port = 3; IP Dst = 10.1.*.*	Forward (2)
Ingress Port = 3; IP Dst = 10.3.*.*	Forward (2)
Ingress Port = 4; IP Dst = 10.1.*.*	Forward (1)
Ingress Port = 4; IP Dst = 10.3.*.*	Forward (1)

Explanation:

- **Clockwise Direction:** From switch s2, the clockwise path follows s2 → s1 → s3. Therefore, traffic from host h3 (Port 3) destined for outside subnets should be sent out through **Port 2** (towards s1).
- **Counter-clockwise Direction:** From switch s2, the counter-clockwise path follows s2 → s3 → s1. Therefore, traffic from host h4 (Port 4) destined for outside subnets should be sent out through **Port 1** (towards s3).

Question 56: SDN OpenFlow Forwarding (Switches s1 and s3)

Consider again the scenario from the previous problem. Give the flow table entries at packet switches s1 and s3, such that any arriving datagrams with a source address of h3 or h4 are routed to the destination hosts specified in the destination address field in the IP datagram.

Hint: Your forwarding table rules should include the cases that an arriving datagram is destined for a directly attached host or should be forwarded to a neighboring router for eventual host delivery there.

Solution

S1 Flow Table

Match	Action
IP Src = 10.2.*.*; IP Dst = 10.1.0.1	Forward (2)
IP Src = 10.2.*.*; IP Dst = 10.1.0.2	Forward (3)
IP Src = 10.2.*.*; IP Dst = 10.3.*.*	Forward (1)

S3 Flow Table

Match	Action
IP Src = 10.2.*.*; IP Dst = 10.3.0.6	Forward (1)
IP Src = 10.2.*.*; IP Dst = 10.3.0.5	Forward (2)
IP Src = 10.2.*.*; IP Dst = 10.1.*.*	Forward (3)

Logical Breakdown: Based on the topology in Figure 4.30:

- **Switch s1:** Directly connects to **h1** (Port 2) and **h2** (Port 3). Port 1 leads to **s3**.
- **Switch s3:** Directly connects to **h6** (Port 1) and **h5** (Port 2). Port 3 leads to **s1**.

Question 57: SDN OpenFlow as a Firewall

Consider again the SDN OpenFlow network shown in Figure 4.30. Suppose we want switch **s2** to function as a firewall. Specify the flow table in **s2** that implements the following firewall behaviors (specify a different flow table for each of the four firewalling behaviors below) for delivery of datagrams destined to **h3** and **h4**. You do not need to specify the forwarding behavior in **s2** that forwards traffic to other routers.

- Only traffic arriving from hosts **h1** and **h6** should be delivered to hosts **h3** or **h4** (i.e., traffic from hosts **h2** and **h5** is blocked).
- Only **TCP** traffic is allowed to be delivered to hosts **h3** or **h4** (i.e., UDP traffic is blocked).
- Only traffic destined to **h3** is to be delivered (i.e., all traffic to **h4** is blocked).
- Only **UDP** traffic from **h1** and destined to **h3** is to be delivered. All other traffic is blocked.

Solution

Under the assumption that OpenFlow blocks any traffic not explicitly matched by a rule (default deny), the flow tables are:

a) Only h1 and h6 to h3 or h4:

Match	Action
IP Src = 10.1.0.1; IP Dst = 10.2.0.3	Forward (3)
IP Src = 10.1.0.1; IP Dst = 10.2.0.4	Forward (4)
IP Src = 10.3.0.6; IP Dst = 10.2.0.3	Forward (3)
IP Src = 10.3.0.6; IP Dst = 10.2.0.4	Forward (4)

b) Only TCP traffic to h3 or h4:

Match	Action
IP Dst = 10.2.0.3; IP Proto = TCP	Forward (3)
IP Dst = 10.2.0.4; IP Proto = TCP	Forward (4)

c) Only traffic to h3 is delivered:

Match	Action
IP Dst = 10.2.0.3	Forward (3)

d) Only UDP traffic from h1 to h3:

Match	Action
IP Src = 10.1.0.1; IP Dst = 10.2.0.3; IP Proto = UDP	Forward (3)